Mini-Projet : Comparaison G-comp, One-step et TMLE

ELIE Kouldjim

2025-12-18

```
library(tidyverse)
library(SuperLearner)
set.seed(12345) # Pour la reproductibilité [cite: 2]
```

1. Introduction

Ce projet vise à comparer trois estimateurs du paramètre $\Psi(P)$ dans un cadre de simulation

-1: L'estimateur de substitution naïve (G-computation)

-2.L'estimateur avec correction en une étape (One-step)

-3.L'estimateur par maximum de vraisemblance ciblée (TMLE)

-4. La valeur théorique cible est $\Psi(P) = 1$

2. Définition du modèle (La Loi P)

```
expit <- plogis # [cite: 9]

simulate <- function(n) {
  W <- rnorm(n, mean = 0, sd = 1) # [cite: 11, 12]
  A <- rbinom(n, 1, expit(-0.2 + W)) # [cite: 13]
  Y <- runif(n,
             min = -2 + abs(W) + A * W^2,
             max = 2 + abs(W) + A * W^2) # [cite: 14, 15, 16]
  obs <- cbind(W = W, A = A, Y = Y) # [cite: 17]
  return(obs)
}
```

3. Algorithmes d'apprentissage (Super Learner)

Nous utilisons une bibliothèque simple pour l'apprentissage des fonctions de nuisance $\bar{Q}_P$ (résultat) et $\bar{g}_P$ (propension).

```r
SL.lib <- c("SL.mean", "SL.glm")

SL.oracular <- function (Y, X, newX, family, obsWeights, model = TRUE, ...) {
  fit.glm <- glm(Y ~ I(abs(W)) + A * I(W^2), data = X, family = family, weights = obsWeights) # [cite: 
  pred <- predict(fit.glm, newdata = newX, type = "response")
  fit <- list(object = fit.glm)
  class(fit) <- "SL.oracular"
  out <- list(pred = pred, fit = fit)
  return(out)
}
predict.SL.oracular <- predict.SL.glm
```

4. Définition des estimateurs

Voici les fonctions d'estimation adaptées du document.

```r
get_gcomp <- function(obs) {
  sl_qbar <- SuperLearner(Y = obs[, "Y"], X = data.frame(W = obs[, "W"], A = obs[, "A"]),
                          family = "gaussian", SL.library = c(SL.lib, "SL.oracular")) # [cite: 126, 130
  qbar_n <- function(A, W) { predict(sl_qbar, newdata = data.frame(W = W, A = A))$pred } # [cite: 131, 
  gcomp <- mean(qbar_n(1, obs[, "W"]) - qbar_n(0, obs[, "W"])) # [cite: 136, 137]
  return(list(estimator = gcomp, qbar_n = qbar_n))
}

get_one_step <- function(obs, qbar_n, threshold = 0.01) {
  sl_gbar <- SuperLearner(Y = obs[, "A"], X = data.frame(W = obs[, "W"]),
                          family = "binomial", SL.library = SL.lib) # [cite: 166, 170]
  gbar_n <- function(W) { as.vector(predict(sl_gbar, newdata = data.frame(W = W))$pred) } # [cite: 171,

  GW <- gbar_n(obs[, "W"]) # [cite: 178]
  gW_obs <- obs[, "A"] * GW + (1 - obs[, "A"]) * (1 - GW) # [cite: 179, 180]
  gW_obs <- pmax(threshold, pmin(1 - threshold, gW_obs)) # [cite: 183]

  correction <- mean(((2 * obs[, "A"] - 1) / gW_obs) * (obs[, "Y"] - qbar_n(obs[, "A"], obs[, "W"]))) #
  one_step <- mean(qbar_n(1, obs[, "W"]) - qbar_n(0, obs[, "W"])) + correction # [cite: 190, 191]
  return(list(estimator = one_step, gbar_n = gbar_n))
}

get_tmle <- function(obs, qbar_n, gbar_n, threshold = 0.01) {
  GW <- gbar_n(obs[, "W"]) # [cite: 228]
  gW_obs <- obs[, "A"] * GW + (1 - obs[, "A"]) * (1 - GW) # [cite: 229, 230]
  gW_obs <- pmax(threshold, pmin(1 - threshold, gW_obs)) # [cite: 231]

  H <- (2 * obs[, "A"] - 1) / gW_obs # [cite: 236, 237]
  fit <- glm(obs[, "Y"] ~ -1 + H + offset(qbar_n(obs[, "A"], obs[, "W"])), family = "gaussian") # [cite
  eps <- coef(fit) # [cite: 241]

  q_star <- function(a, w) {
    ga <- a * gbar_n(w) + (1 - a) * (1 - gbar_n(w)) # [cite: 244]
    qbar_n(a, w) + eps * (2 * a - 1) / pmax(threshold, ga) # [cite: 248, 249]
  }
  tmle <- mean(q_star(1, obs[, "W"]) - q_star(0, obs[, "W"])) # [cite: 251, 253]
```

```
  return(tmle)
}
```

5. Étude de Simulation

Nous comparons les résultats pour B = 50 répétitions avec deux tailles de population.

```
B <- 50
tailles_n <- c(50, 250)
resultats_liste <- list()

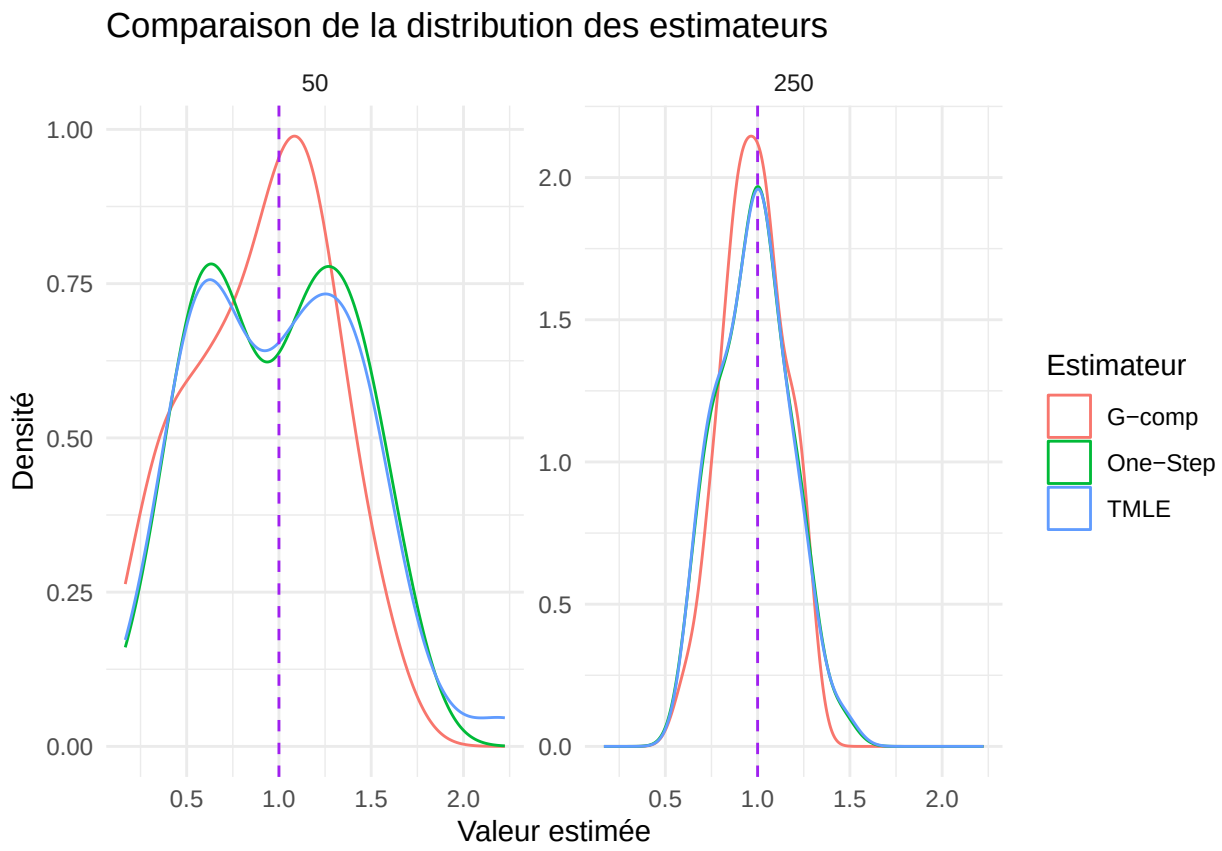
for (n in tailles_n) {
  ESTIMATES <- matrix(NA, nrow = B, ncol = 3)
  colnames(ESTIMATES) <- c("G-comp", "One-Step", "TMLE")

  for (bb in 1:B) {
    data_sim <- simulate(n)
    fit_g <- get_gcomp(data_sim)
    fit_os <- get_one_step(data_sim, fit_g$qbar_n)
    fit_tmle <- get_tmle(data_sim, fit_g$qbar_n, fit_os$gbar_n)
    ESTIMATES[bb, ] <- c(fit_g$estimator, fit_os$estimator, fit_tmle)
  }
  resultats_liste[[as.character(n)]] <- as.data.frame(ESTIMATES) %>% mutate(n = n)
}
df_final <- bind_rows(resultats_liste) %>% pivot_longer(cols = 1:3, names_to = "Estimateur", values_to =
```

6. Analyse des Résultats

Le graphique ci-dessous montre la distribution des estimations. La ligne verticale violette représente la valeur réelle $\Psi(P) = 1$.

```
ggplot(df_final, aes(x = Valeur, color = Estimateur)) +
  geom_density() +
  geom_vline(xintercept = 1, color = "purple", linetype = "dashed") +
  facet_wrap(~n, scales = "free_y") +
  theme_minimal() +
  labs(title = "Comparaison de la distribution des estimateurs", x = "Valeur estimée", y = "Densité")
```




```
df_final %>%
  group_by(n, Estimateur) %>%
  summarise(Biais = mean(Valeur) - 1,
            MSE = mean((Valeur - 1)^2), .groups = 'drop') %>%
  knitr::kable(digits = 4)
```

n	Estimateur	Biais	MSE
50	G-comp	-0.0958	0.1453
50	One-Step	-0.0194	0.1634
50	TMLE	-0.0063	0.1893
250	G-comp	-0.0214	0.0277
250	One-Step	-0.0215	0.0362
250	TMLE	-0.0224	0.0364